

Object-Oriented Programming in Java

Lecture 1 - Organization and Introduction

Emily Lucia Antosch

HAW Hamburg

19.10.2025

Contents

1. Organization	2
2. Introduction	11
3. The Java Programming Language	28
4. The First Program	38
5. Literature	46
6. License Notice	48

1. Organization

1.1 The Goal of This Chapter

1. Organization

- I want to introduce myself to you and discuss the course structure of this module.
- You will get an overview of the prerequisites for this module and will be able to meet them.
- You will know how to reach me.

- Emily Lucia Antosch, 25 years old
- Bachelor's degree in Electrical and Information Technology
- Backend Software Engineer at MOIA
- Currently doing my Master's degree in Practical Computer Science
- Email: emilylucia.antosch@haw-hamburg.de

i Info

I'm rather new with teaching at university, so please be lenient with me!

- Lectures are on Wednesday between 15:55 and 19:15.
 - ▶ At the beginning, there are many sessions designed to prepare you for the lab.
- I would ask you to actively participate in the lectures.
- There will be small questions and tasks that you can answer live and code along with.

! Memorize

If you don't understand something, please ask immediately!
I'm more than happy to repeat any part of the lecture!

1.4 Laboratory Sessions

1. Organization

- There will be **4 labs** throughout the semester.
- **Successful participation in the labs is mandatory** for taking the exam.
- You will work in **teams of two**.
- Lab selection happens in **myHAW**.



Warning

Without successful lab participation, you cannot take the exam!

1.5 Lab Preparation and Participation

1. Organization

- Each lab requires you to **prepare the tasks beforehand**.
- If you struggle with some tasks:
 - ▶ You still need to understand what you did and what the task entails.
 - ▶ Lab lecturers will help you complete tasks you couldn't finish.

! Memorize

Preparation is key! Come to the lab with questions, not blank pages.

1.6 Lab Presentations

1. Organization

- You must **present at least two tasks** during the semester to your lab class.
- Important points:
 - ▶ You do **not** have full authority over which tasks you present.
 - ▶ **Don't wait** until the last lab to present both tasks.
 - ▶ Be prepared to present any task you've worked on.



Tip

Start presenting early in the semester to reduce pressure later!

- We want to build upon your prior knowledge.
- By the end of the lecture, you should be able to create simple programs in Java.
- Also, we want to look at mastering object-oriented programming paradigm and be able to highlight the differences to other programming paradigms.
- You can also read about the exact content outside of the lecture in the module handbook.

- You will need an installation of the **Java SDK**.
 - ▶ For this, I have written a guide that you can find in the Moodle room.
- Additionally, the lecture will use the tool **JetBrains IntelliJ**.
 - ▶ This is, in my opinion, a very good and simple IDE for beginners.

2. Introduction

2.1 The Goal of This Chapter

- You will be able to apply your existing knowledge from previous lectures to new content.
- You will understand the fundamental concepts of object-oriented programming and understand the difference from programming in C.
- We will create a simple program in the IntelliJ IDEA development environment and execute it.

2.2 Topic Overview: Fundamentals

The first lectures focus on the following principles:

1. Imperative Concepts
2. Classes and Objects
3. Class Library
4. Inheritance
5. Interfaces

2.3 Topic Overview: Advanced Concepts

From the fundamentals, we then want to derive further concepts:

- 6. Graphical User Interfaces
- 7. Exception Handling
- 8. Input and Output
- 9. Multithreading (Parallel Computing)

2.4 Objects and Classes

In the real world, things are often determined and described by their properties:

- A car has properties such as
 - ▶ a manufacturer
 - ▶ a color
 - ▶ fuel consumption



Idea

Using object-oriented programming, we can apply this intuitive approach to programming as well!

? Question

What properties could you use to describe a person, for example? How might this fit into the programming context?

? Question

What properties could you use to describe a person, for example? How might this fit into the programming context?

- For students:
 - ▶ Name, address, student ID number
- For programs/websites:
 - ▶ Username, password, join date

2.4 Objects and Classes

- To create multiple similar objects from this similar blueprint, a class is created:
 - ▶ It contains all the properties we just defined in variables.
 - ▶ From it, completely different objects can be created that have these properties filled in differently.



Example

From the **Student** class, for example, the two students **Max** and **Ines** can be created, who both have different names and their own student ID number.

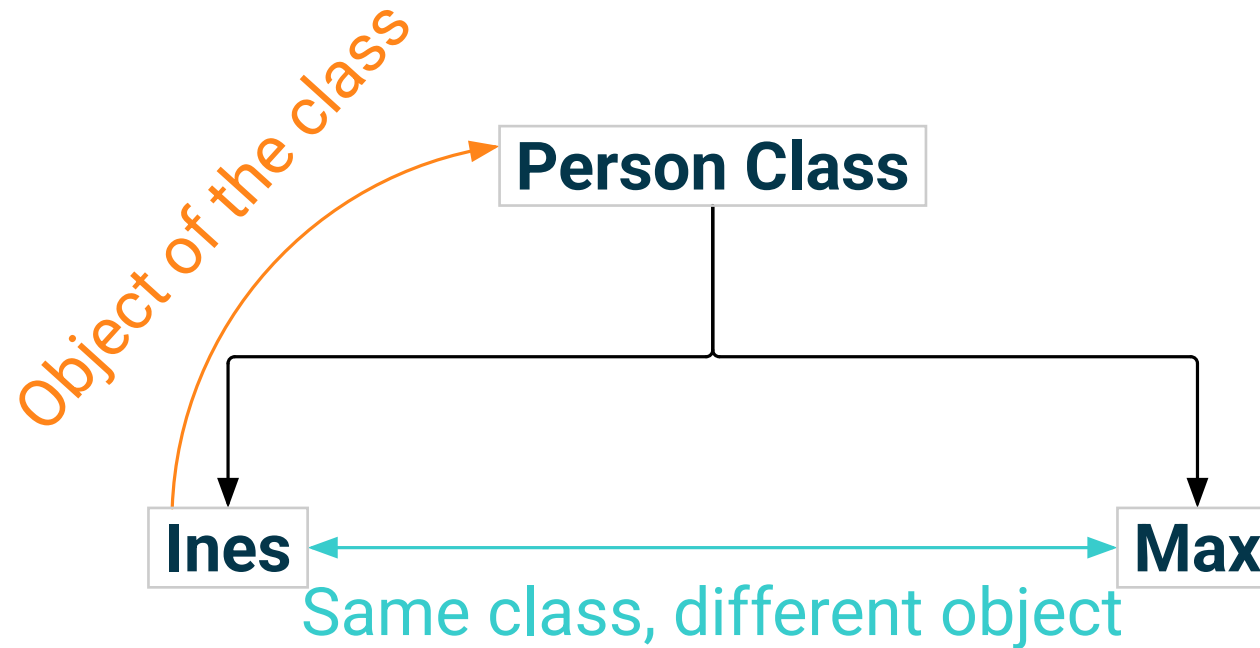


Figure 1: Relationship between classes and objects of that class

2.4 Objects and Classes

- Variables and functions are thus combined into a class.
 - ▶ A set of variables is defined.
 - ▶ For these variables, functions are introduced that can read and modify them.

! Memorize

- Variables are called **attributes**.
- The values of these variables describe the **state** of the object.
- Functions are called **methods**.

2.4 Objects and Classes

- So-called UML class diagrams can be used to describe classes with their **attributes** and **methods**.

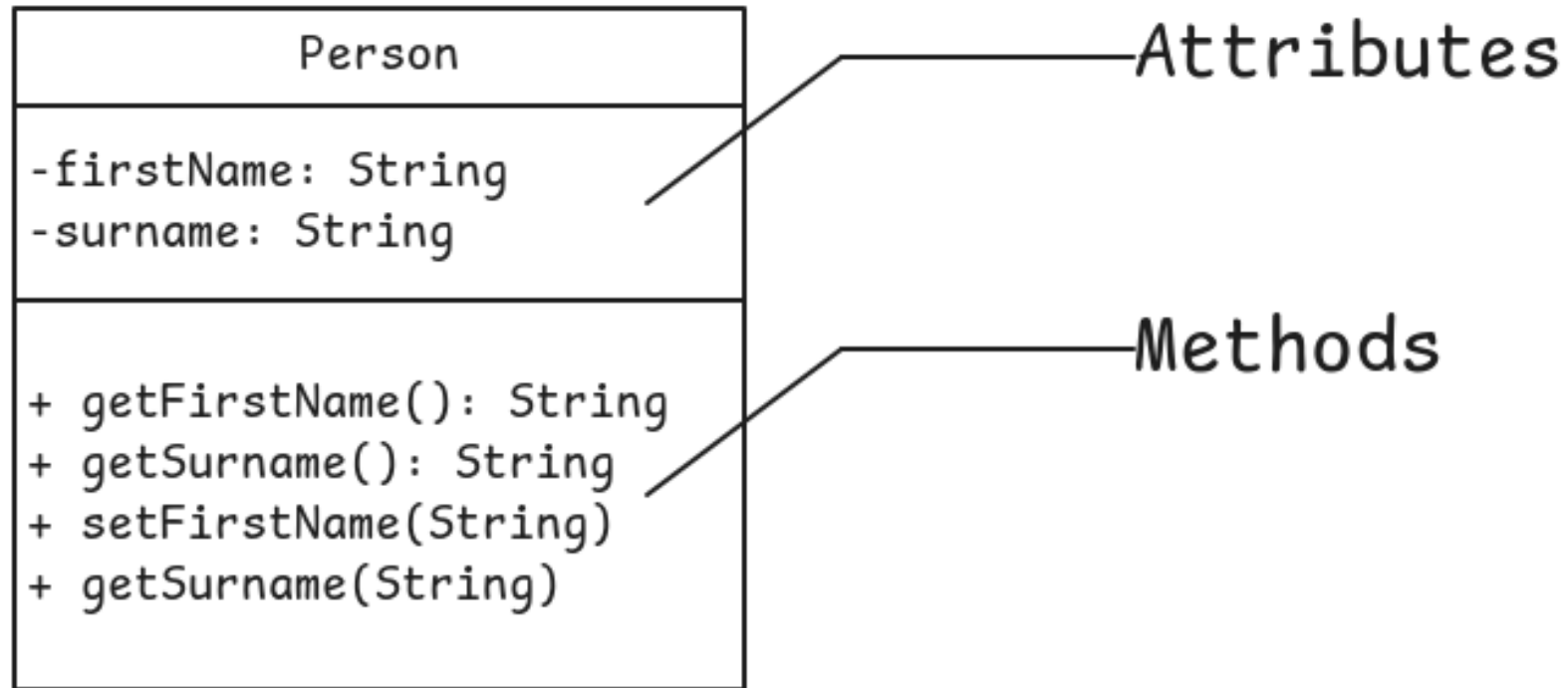


Figure 2: UML Class Diagram

2.5 Data Encapsulation

- Data can be encapsulated using attributes and methods of classes.

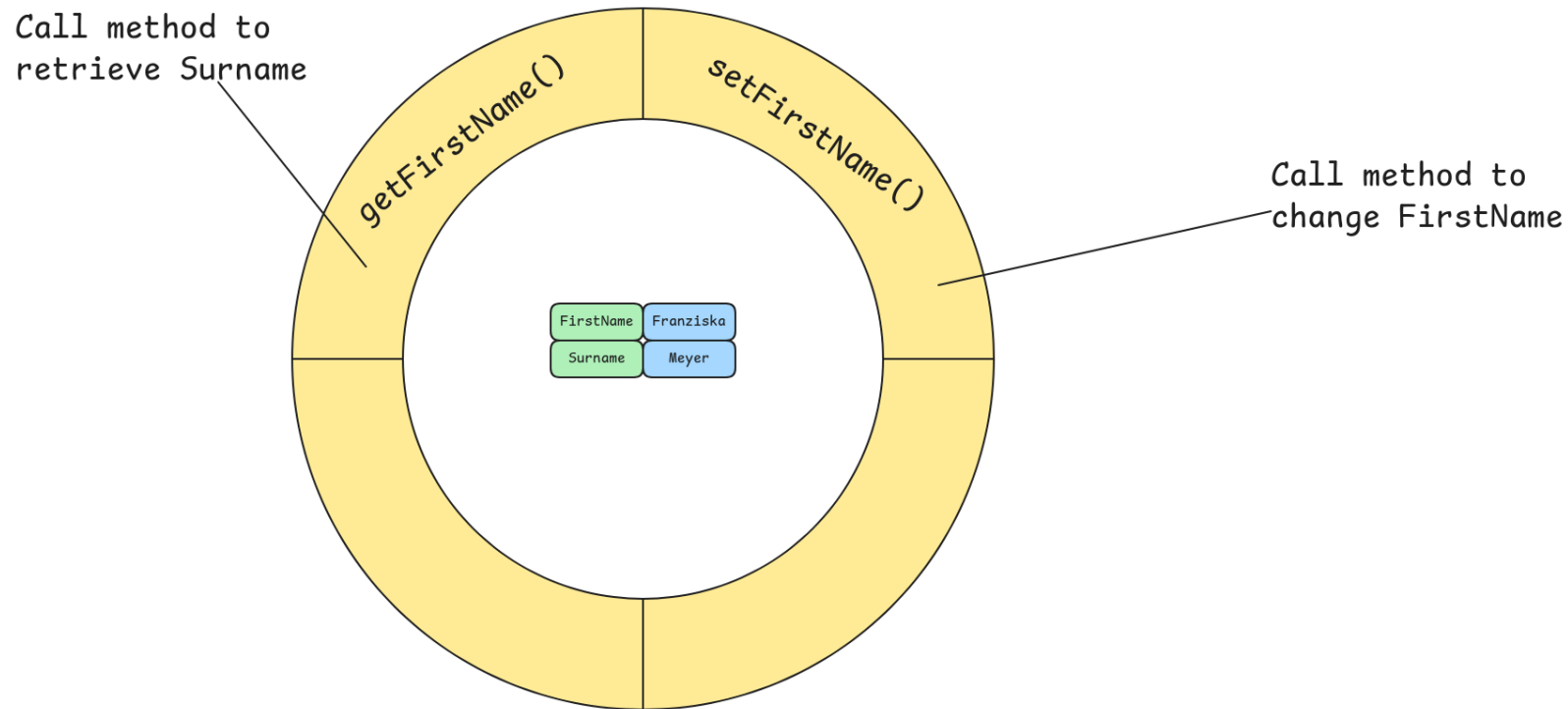


Figure 3: Data encapsulation through classes

2.5 Data Encapsulation

- Data can be encapsulated using attributes and methods of classes.
 - ▶ Not all parts of the program can access encapsulated data, which increases security.
 - ▶ Additionally, attributes can be protected from erroneous values this way.

? Question

Where do the differences lie in comparison to the C programming language?

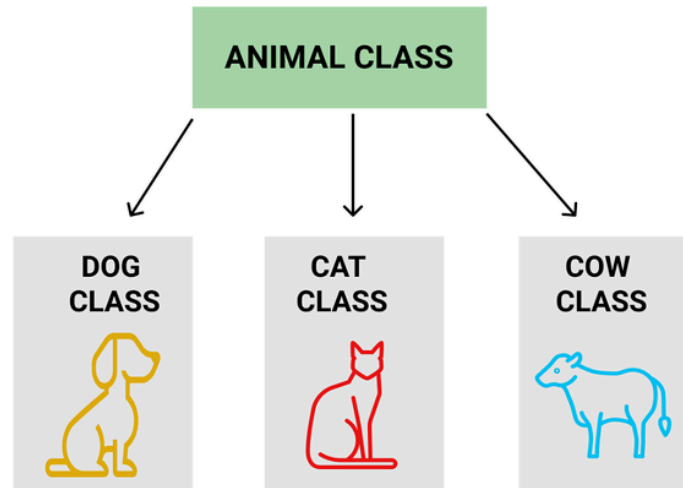
? Question

Where do the differences lie in comparison to the C programming language?

- The data structure (i.e., the **struct**) must be made public for access to the elements.
- The data is not protected.
- There is no association between the data and the functions.

2.6 Inheritance

- Through inheritance, new classes can be created from other classes.
 - ▶ The methods and attributes of the base class are inherited and extended with additional code.
 - ▶ No duplicated code.



2.7 Composition

- Classes can also be composed of other classes.
- This is called **composition**.
- For example, the **House** class would be composed of **Windows**, **Walls**, and **Doors**.

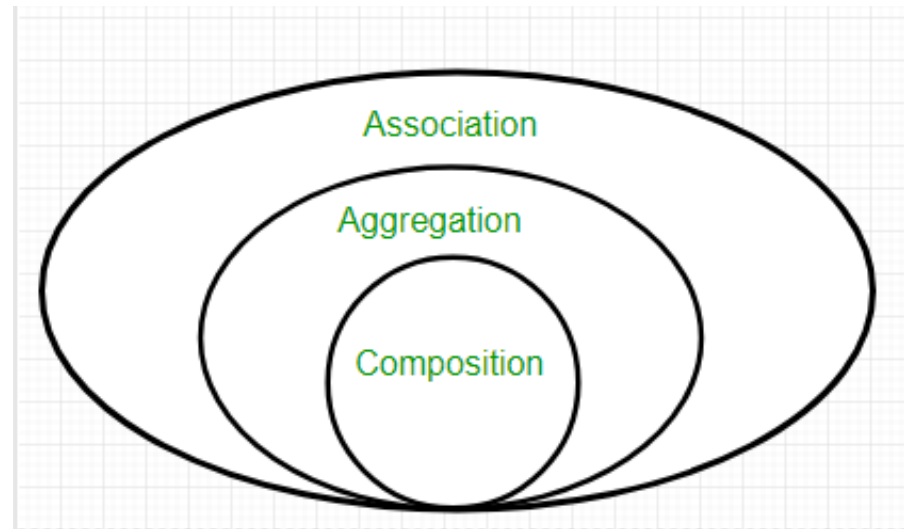


Figure 5: Composition of a given class

2.8 Program Execution in Java

1. At program startup, a special **main** method is executed in the *main object*.
2. In this method, objects are created and the **references** to these objects are stored in variables.
3. These variables can then be used to access the respective object.
4. Objects in the program can then create additional objects and call methods.
5. As soon as the **main** method is finished, the program ends.

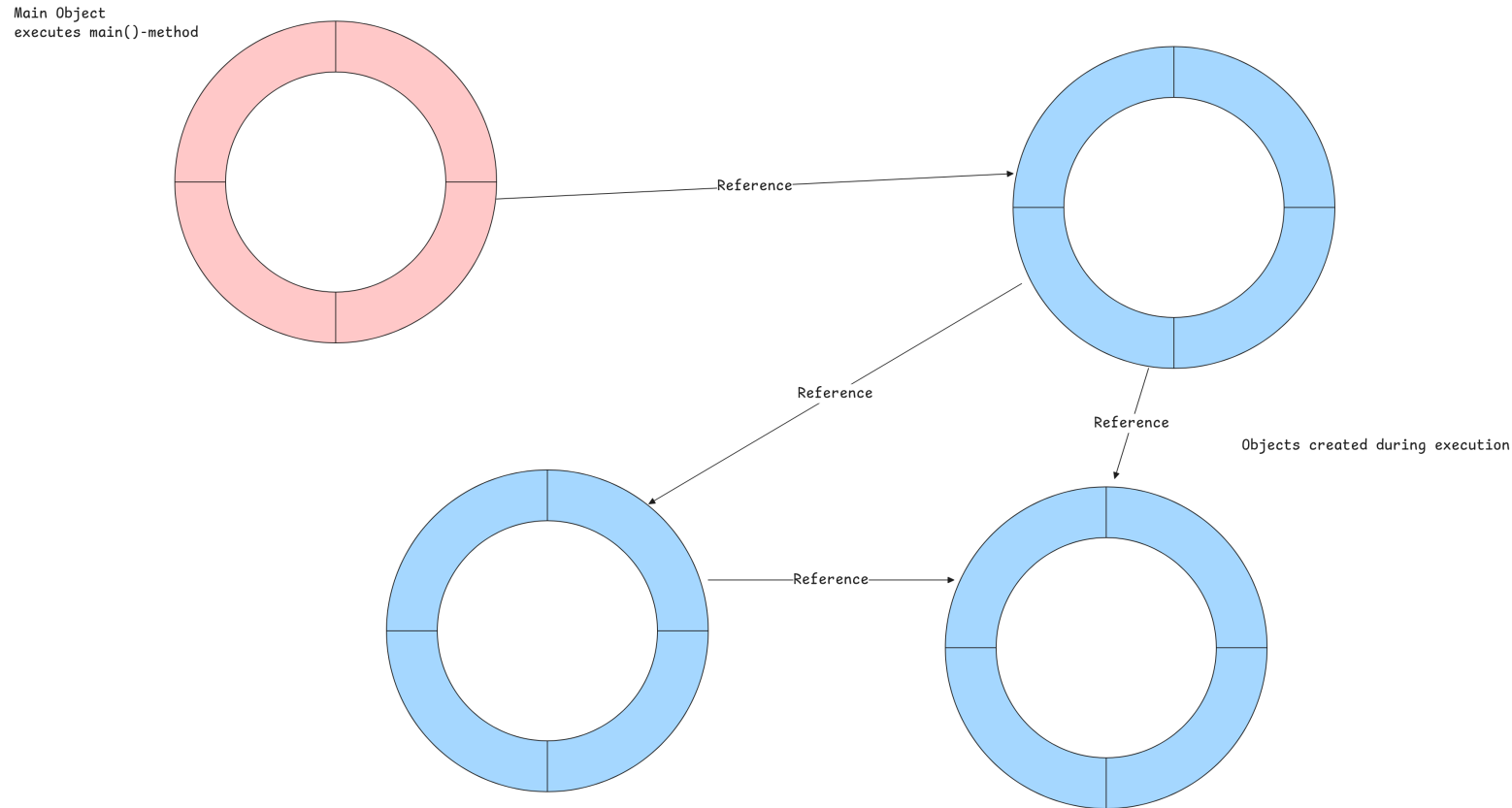


Figure 6: References in the lifetime of a program

3. The Java Programming Language

☰ Task 1

Let's first write a few simple tasks in the programming language C:

- Sum of numbers 1 to n using a **for** loop.
- Maximum of two numbers using an **if** statement.
- Determine the maximum of two numbers using the **getMax()** function.

3.1 Java vs. C

3. The Java Programming Language

- I have good news: You could have executed this code in Java without any problems!
- The **syntax**, i.e., the keywords and structure of the language, is very close to C and C++!
- Therefore, we want to continue building on your prior knowledge.

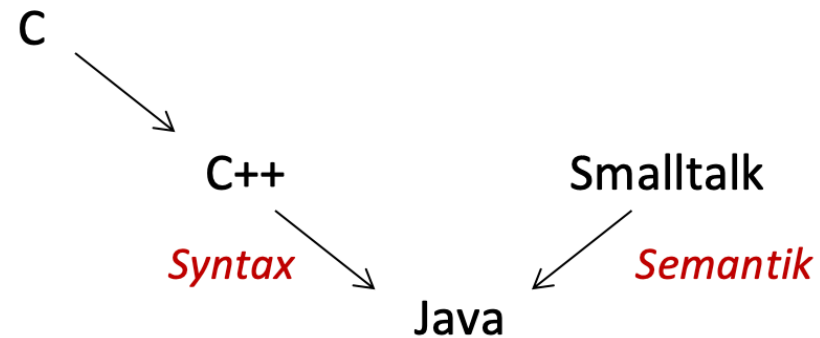


Figure 7: The influences on the Java programming language

3.2 Compilation

3. The Java Programming Language

1. Development

- Source code is written on the PC.
- Compiler compiles source code into **bytecode**.

2. Execution

- Bytecode is executed on the **JVM** (Java Virtual Machine).
- Execution does not require recompilation for each target platform.

3.2 Compilation

3. The Java Programming Language

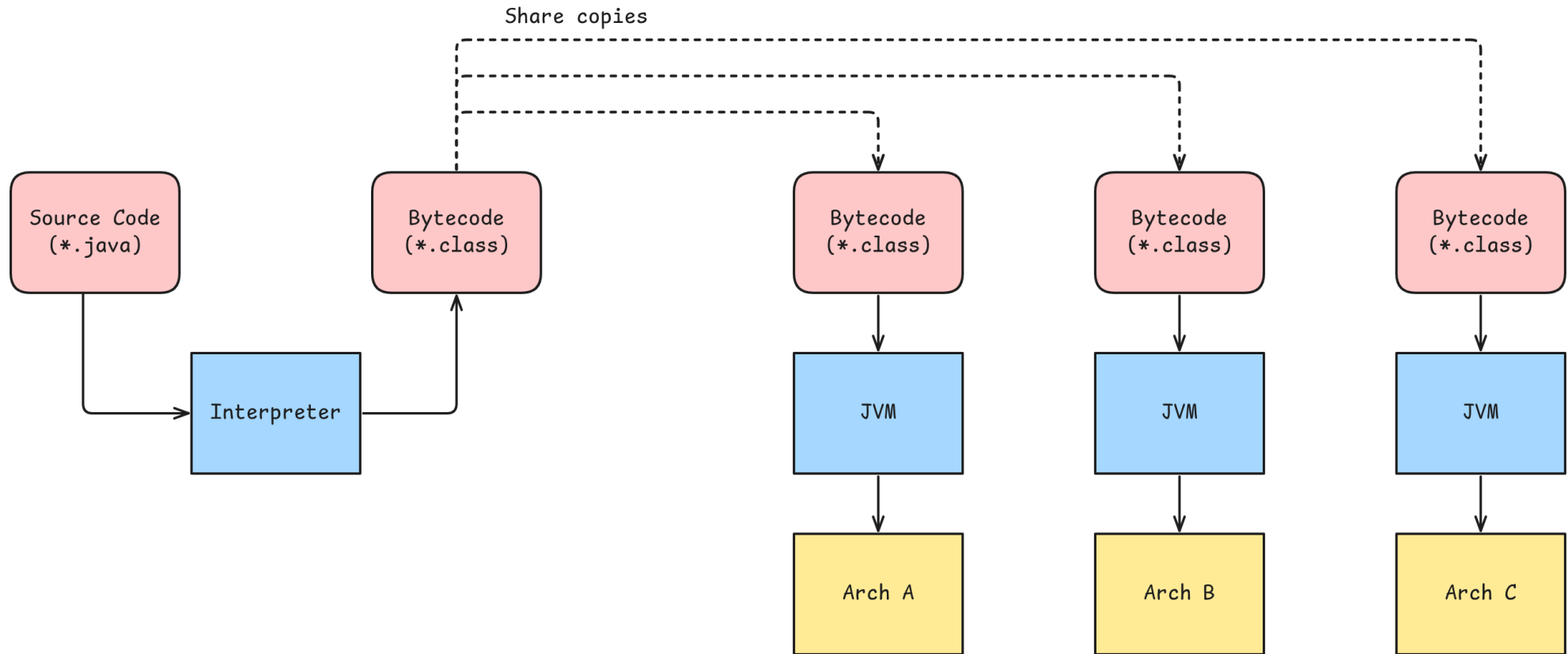


Figure 8: Program execution with the JVM

3.2 Compilation

3. The Java Programming Language

- Differences in other programming languages that are compiled or interpreted:
 - ▶ **Compiled languages** must be recompiled for each target platform.
 - ▶ **Interpreted languages** must be interpreted by their own interpreter on the target platform itself.

3.2 Compilation

3. The Java Programming Language

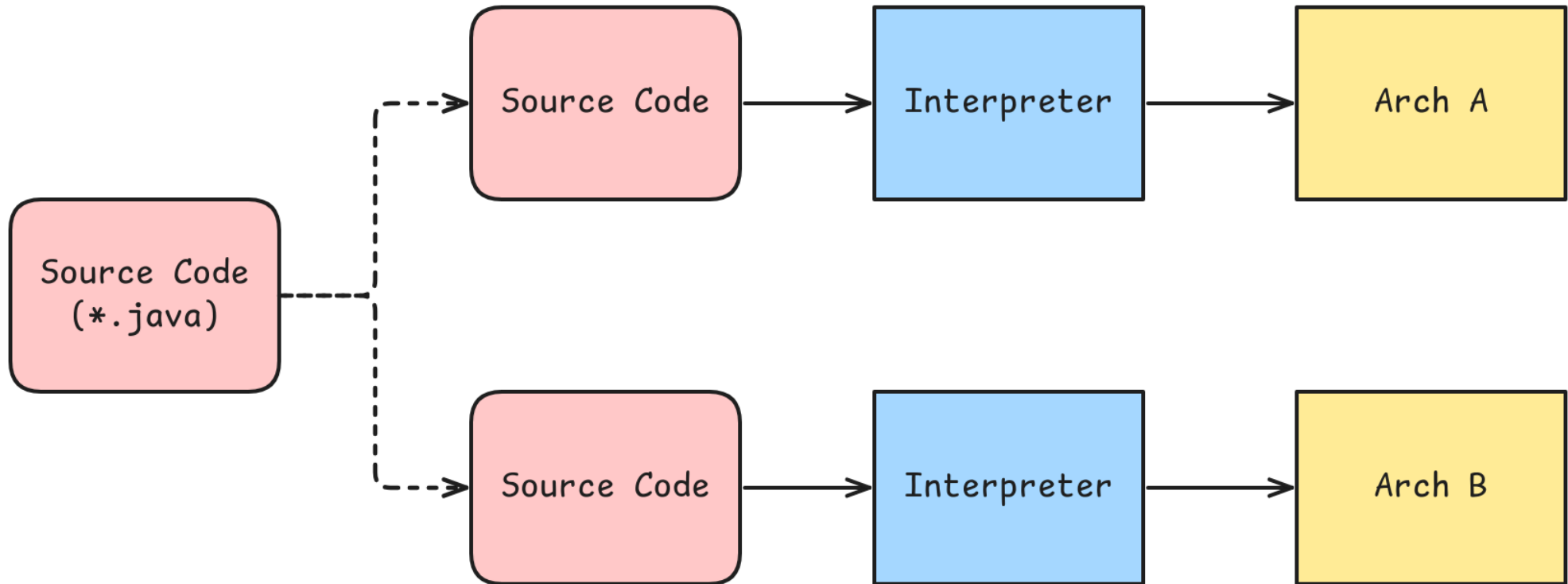


Figure 9: Execution of compiled languages

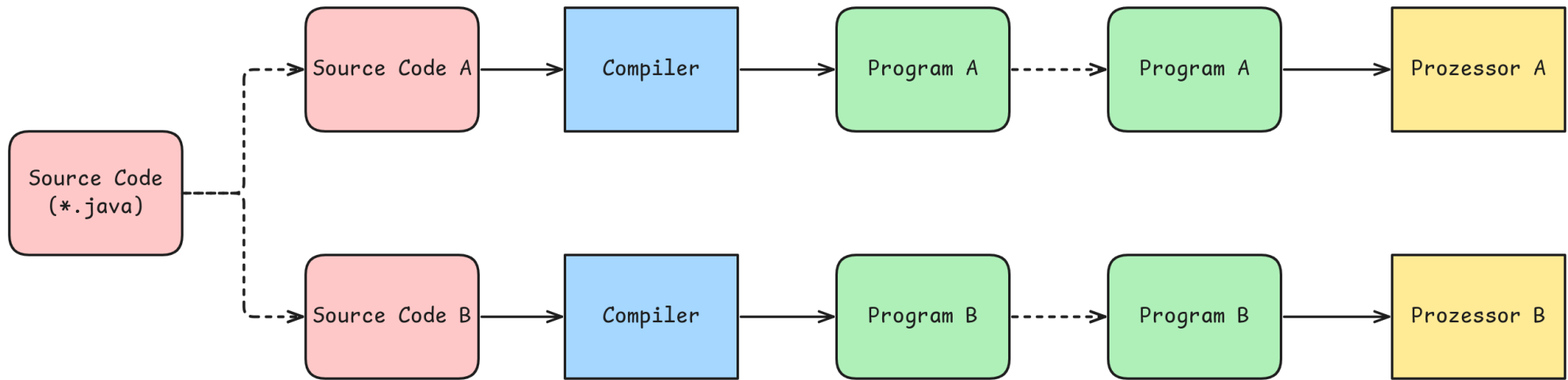


Figure 10: Execution of interpreted languages

? Question

If you could develop a new language, what would be important to you? What would you change about C/C++?

? Question

If you could develop a new language, what would be important to you? What would you change about C/C++?

- Java
 - ▶ Object-oriented language (i.e., classes, objects, and inheritance)
 - ▶ Platform-independent (via **JVM**)
 - ▶ Strongly typed (fixed types like **int** and **String**)
 - ▶ Robust (i.e., Garbage Collector)

? Question

Which is the better programming language: C or Java?

4. The First Program

- I would recommend IntelliJ IDEA from JetBrains as an IDE.
 - ▶ This tool will also be used in the exam.
 - ▶ The IDE also includes the Java JDK, which you need for programming.



- Choose the Community Edition at <https://www.jetbrains.com/idea/download/?section=windows>, or scan the QR code.

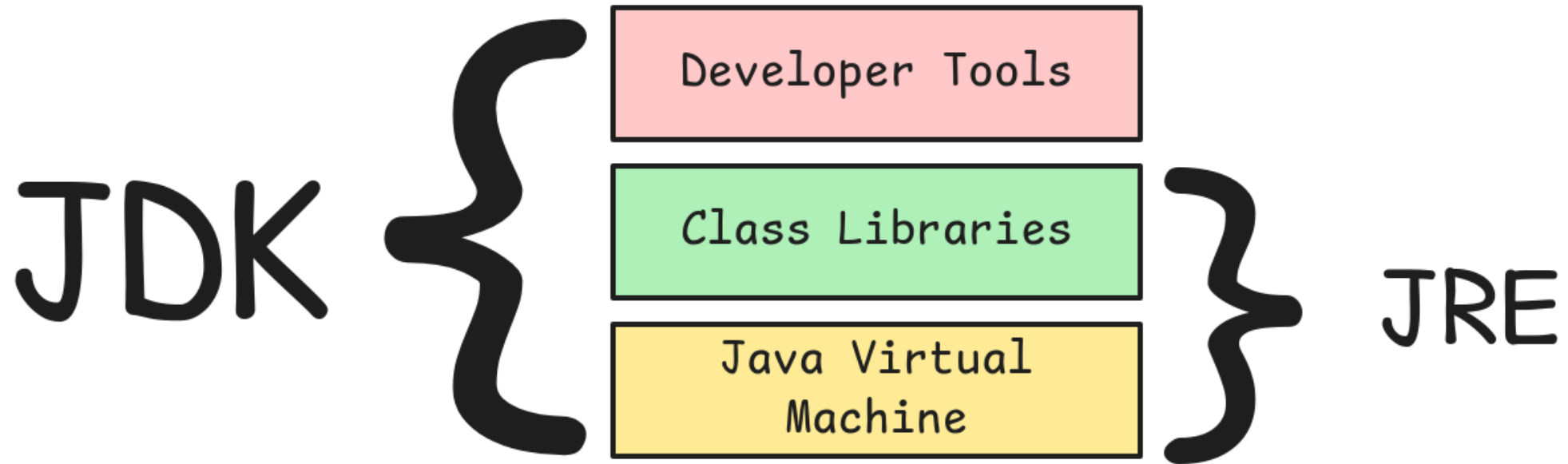


Figure 8: The structure of the Java Toolchain

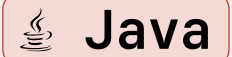
Task 2

- Preparation:
 1. First open a directory where you will store the programming files.
 2. Open IntelliJ IDEA.
- Create project:
 1. Select File > New > Project.
 2. Assign a name and a location.
 3. Choose Java and IntelliJ and the appropriate JDK
 4. Click “Create”

Task 3

- Create package
 1. Right-click on src
 2. Select New > Package
 3. Enter name
- Create class
 1. Right-click on package
 2. Choose New > Java Class

```
1 public static void main(String[] args){  
2     System.out.println("Hello World!");  
3 }
```

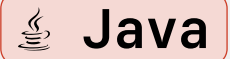


Idea

Enter the code into the file you just created. If you're already that far, feel free to code along!

- A Java file can be executed if it has a public (**public**) class:
`public class MyApplication {...}`
- The class must also have the same name as the file, for example `MyApplication.java`
- The class has the method: `public static void main(String[] args)`

```
1  public class MyApplication {  
2      public static void main(String[] args) {  
3          System.out.println("Hello World!");  
4      }  
5  }
```



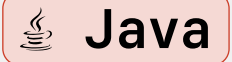
This name is freely selectable.

```
1  public class MyApplication {  
2      public static void main(String[] args) {  
3          System.out.println("Hello World!");  
4      }  
5  }
```



This name is freely selectable.

```
1  public class MyApplication {  
2      public static void main(String[] args) {  
3          System.out.println("Hello World!");  
4      }  
5  }
```



This method must always be called main

5. Literature

- Some books that might help you during the course:
 - ▶ D. Abts: Grundkurs JAVA, Springer-Vieweg
 - ▶ H.-P. Habelitz: Programmieren lernen mit Java, Rheinwerk Computing

6. License Notice

6.1 Attribution

- This work is shared under the CC BY-NC-SA 4.0 License and the respective Public License
- <https://creativecommons.org/licenses/by-nc-sa/4.0/>
- This work is based off of the work Prof. Dr. Marc Hensel.
- Some of the images and texts, as well as the layout were changed.
- The base material was supplied in private, therefore the link to the source cannot be shared with the audience.